



Big Databases : NoSQL





Before all :

let's introduce ourselves

- Me :
 - Training
 - Work experience
- You :
 - Initial training
 - Work/internship experience
 - Level of knowledge in databases, NoSQL, Big data, docker
 - Expectations for this course



Objectives

- 1 : Conceptual understanding of NoSQL databases
- 2 : Proficiency in setting up experimental environment for big databases
- 3 : Master Core NoSQL Databases
- 4 : NoSQL databases integration and choosing the right tool
- 5 : Implement Real-World Data Solutions



Objective 1 : Conceptual understanding of NoSQL databases

- The evolution of data
- Definition of big data
- Traditional databases (RDBMS) limitations
- Introduction to NoSQL Databases



Objective 2 : Proficiency in setting up experimental environment for big databases

- Introduction to containerization
- Docker for Database Environments



Objective 3 : Master Core NoSQL Databases

- Key-Value Stores
- Document Stores
- Column-Family Stores
- Graph Databases



Objective 4 : NoSQL databases integration and choosing the right tool

- When to use each type of NoSQL database?
- Data ingestion strategies (ETL/ELT)
- Introduction to Spark & NoSQL databases



Objective 5 : Implement Real-World Data Solutions

- Presentation and implementation of a global use case



Evaluation methods





Evaluations

- A series of application exercises to be completed after lessons no. 3 and 7
- Participation grade
- Final project : last day



My rules

- => Avoid Chatter : talk about the course, ask questions!
- => No wrong question/answer
- => Give your thoughts, share your knowledge
- => **DO NOT USE CHATGPT** or any other AI => try by yourself first : penalties for those who use it



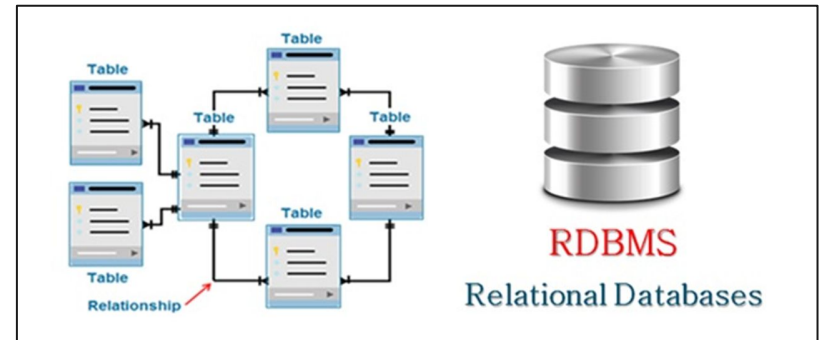
Part 1 :

Big Data Concepts and Challenges

The evolution of data: from structured to big data

Structured data

- Organized in tables (rows and columns)
- Stored in relational databases
- Data managed with SQL in relational DBMSs
- Examples of DBMS :
 - MySQL
 - PostgreSQL
 - Oracle





The evolution of data: from structured to big data

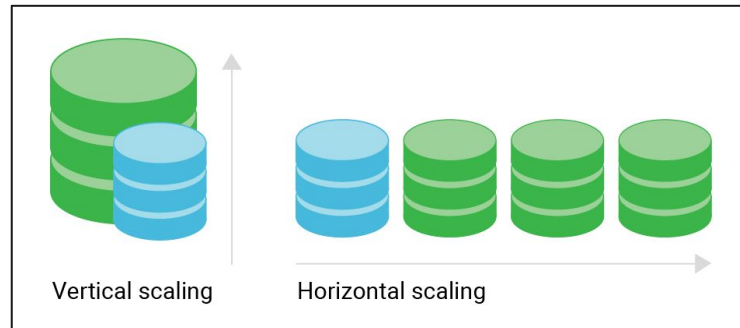
Structured data

- Advantages of relational DBMS :
 - Enforces strong schema (data types, constraints)
 - Types : String, Integer, Boolean, Date, ...
 - Relations : PK, Unique, Non NULL, FK, ...
 - Easy to query with SQL
 - ACID Transactions :
 - **Atomicity** : A transaction must be fully completed or not executed at all; **All or Nothing**
 - **Consistency** : Transactions must bring the DB from one valid state to another; **Always Valid State**
 - **Isolation** : Multiple transactions can occur simultaneously without interfering with each other; **No Cross-Talk**
 - **Durability** : Once a transaction is committed, it will persist even if there's a crash; **Persistence Guaranteed**

The evolution of data: from structured to big data

Structured data

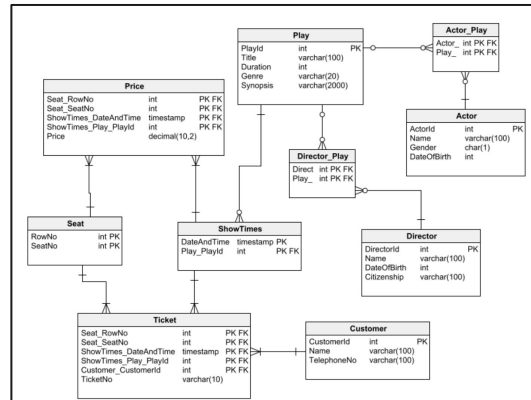
- Limitations :
 - **Scalability** : only optimized for vertical scalability (adding more power to a single server) and not for horizontal scalability (adding more servers)
=> No possibility to use “sharding” (splitting data across servers)



The evolution of data: from structured to big data

Structured data

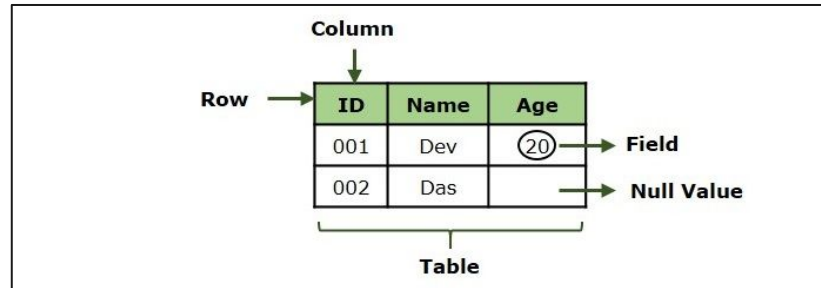
- Limitations :
 - **Schema rigidity** : Adding or modifying columns in production often requires downtime or complex migration scripts
 - Constraints : type, relations ...



The evolution of data: from structured to big data

Structured data

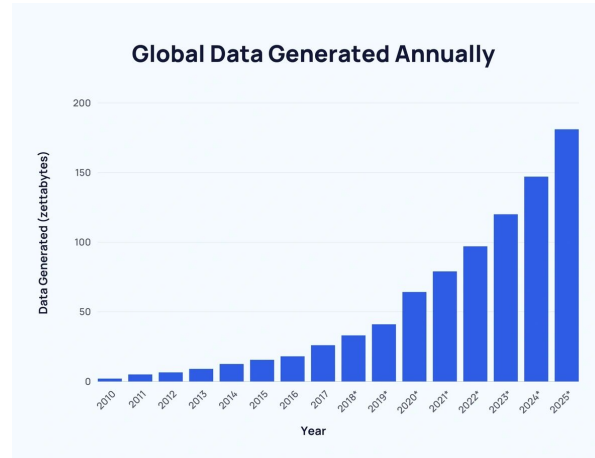
- Limitations :
 - **Inflexibility** : Not optimized for semi-structured, unstructured, hierarchical or high-velocity (real-time) data
 - Images, Audio, Documents, Videos, JSON...



The evolution of data: from structured to big data

Big Data Era

- Explosion of data from:
 - Social media



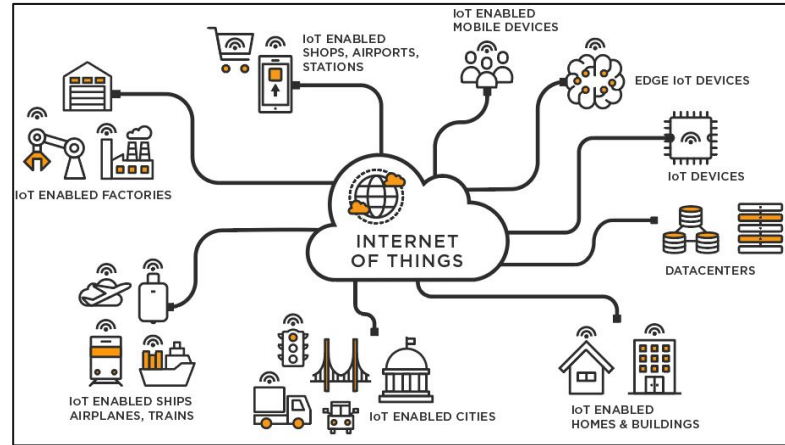
Multiples of bytes				v · d · e
SI decimal prefixes		Binary	IEC binary prefixes	
Name (Symbol)	Value	usage	Name (Symbol)	Value
kilobyte (kB)	10 ³	2 ¹⁰	kibibyte (KiB)	2 ¹⁰
megabyte (MB)	10 ⁶	2 ²⁰	mebibyte (MiB)	2 ²⁰
gigabyte (GB)	10 ⁹	2 ³⁰	gibibyte (GiB)	2 ³⁰
terabyte (TB)	10 ¹²	2 ⁴⁰	tebibyte (TiB)	2 ⁴⁰
petabyte (PB)	10 ¹⁵	2 ⁵⁰	pebibyte (PiB)	2 ⁵⁰
exabyte (EB)	10 ¹⁸	2 ⁶⁰	exbibyte (EiB)	2 ⁶⁰
zettabyte (ZB)	10 ²¹	2 ⁷⁰	zebibyte (ZiB)	2 ⁷⁰
yottabyte (YB)	10 ²⁴	2 ⁸⁰	yobibyte (YiB)	2 ⁸⁰

See also: Multiples of bits · Orders of magnitude of data

The evolution of data: from structured to big data

Big Data Era

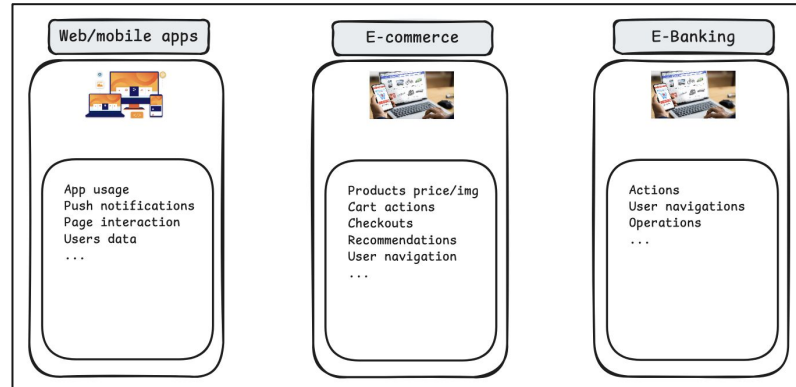
- Explosion of data from:
 - IoT



The evolution of data: from structured to big data

Big Data Era

- Explosion of data from:
 - Logs and clickstreams



Require new models for storage & processing : real time, various formats, ...



The evolution of data: from structured to big data

Characteristics of Big Data

- **Volume** – Massive data size (terabytes to petabytes)
- **Velocity** – Data generated in real-time
- **Variety** – Structured, semi-structured, unstructured
- **Veracity** – Quality & uncertainty of data
- **Value** – Turning data into insights

Transition to NoSQL

Types of Modern Data

Structured



Excel sheet

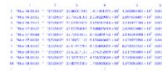
Semi-Structured



JSON



XML

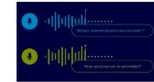


CSV

Unstructured



Images



Audio/Videos



Text



Transition to NoSQL

Concept of NoSQL

- NoSQL ⇔ Not Only SQL
- Databases that allows you to manage the limitations of RDBMS :
 - **Flexible schema** : can store different data types
 - **Horizontal scalability** : Easily scale out by adding more servers
 - **High performance for specific workloads** : massive data, real time
 - **Availability and partition tolerance** : distribute data over servers



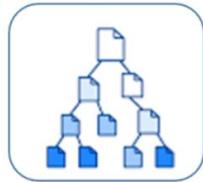
Part 2 :

Introduction to NoSQL Databases

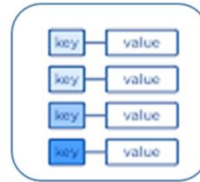
NoSQL Databases

Classification

Store semi-structured data as documents (typically JSON, XML....)
Use cases : user profiles, product catalogs
Ex. : MongoDB, DocumentDB



Document Store



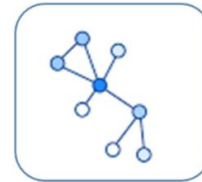
Key-Value Store

Simple data stores using unique keys to access values
Use cases : Caching, shopping carts
Ex. : Redis, DynamoDB



Wide-Column Store

Store data in column families that can be distributed across nodes
Use cases : event logging, IoT applications
Ex. : Cassandra, HBase



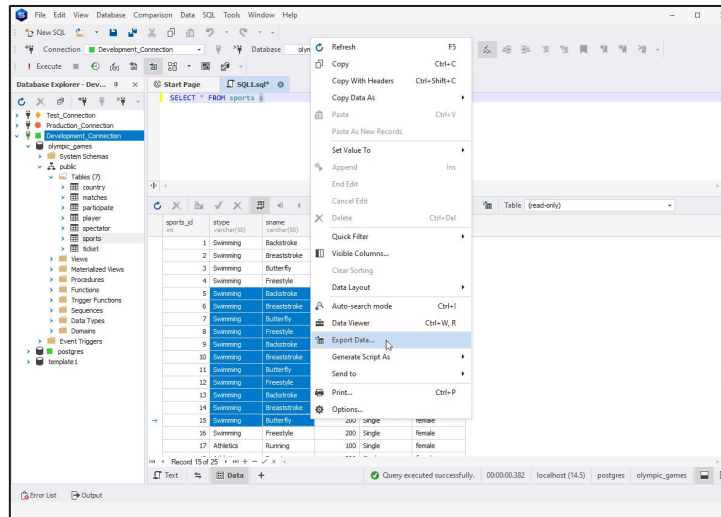
Graph Store

Specialized for highly connected data with explicit relationship modeling
Use cases : Social networks, recommendation engines
Ex. : Neo4j, JanusGraph

NoSQL Databases

Classification

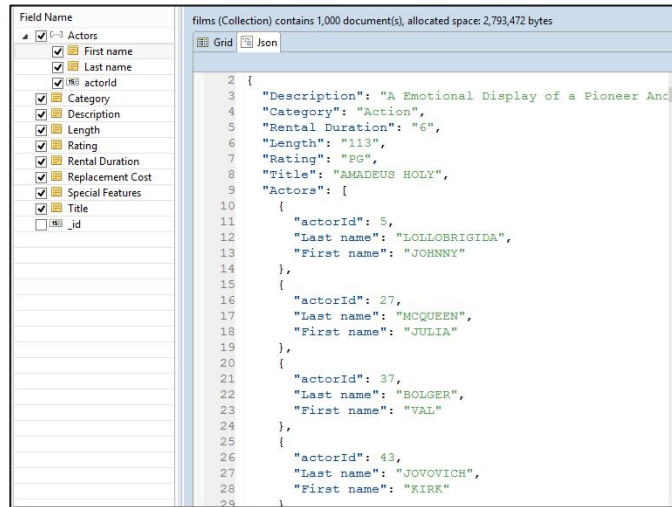
- Relational databases :



NoSQL Databases

Classification

- Document databases :



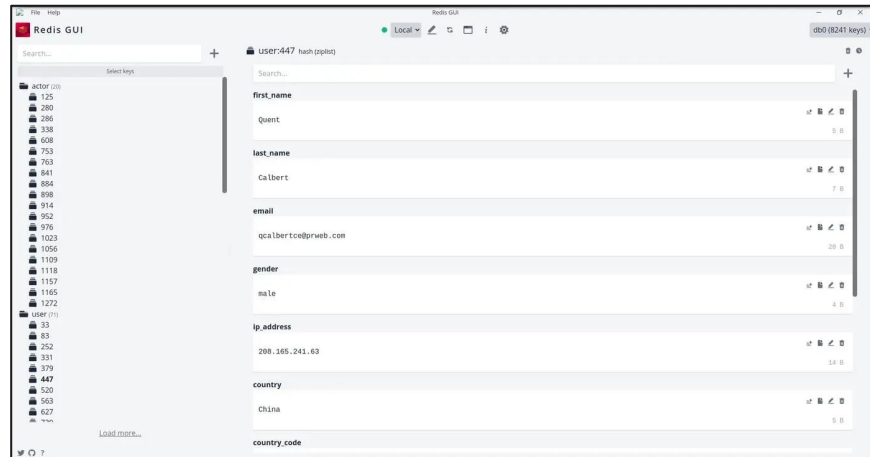
The screenshot displays a NoSQL database interface. On the left, a 'Field Name' list shows various fields with checkboxes, including 'First name', 'Last name', 'actorId', 'Category', 'Description', 'Length', 'Rating', 'Rental Duration', 'Replacement Cost', 'Special Features', 'Title', and '_id'. The main area shows a document in a collection named 'films'. The document is displayed in a grid view and contains the following JSON structure:

```
{
  "Description": "A Emotional Display of a Pioneer And",
  "Category": "Action",
  "Rental Duration": "6",
  "Length": "113",
  "Rating": "PG",
  "Title": "AMADEUS HOLY",
  "Actors": [
    {
      "actorId": 5,
      "Last name": "LOLLOBRIGIDA",
      "First name": "JOHNNY"
    },
    {
      "actorId": 27,
      "Last name": "MCQUEEN",
      "First name": "JULIA"
    },
    {
      "actorId": 37,
      "Last name": "BOLGER",
      "First name": "VAL"
    },
    {
      "actorId": 43,
      "Last name": "JOVOVICH",
      "First name": "KIRK"
    }
  ]
}
```

NoSQL Databases

Classification

- Key-value databases :



NoSQL Databases

Classification

- Column-family databases :

Each cell has multiple versions, typically represented by the timestamp of when they were inserted into the table

Timestamp1 Timestamp2

Row Key	Column Family - Personal		Column Family - Office		Address
	Name	Residence Phone	Phone	Phone	
00001	John	415-111-1234	415-212-5544		1021 Market St
00002	Paul	408-432-9922	415-212-5544		1021 Market St
00003	Ron	415-993-2124	415-212-5544		1021 Market St
00004	Rob	815-243-9988	408-998-4322		4455 Bird Ave
00005	Carly	206-221-9123	408-998-4325		4455 Bird Ave
00006	Scott	818-231-2566	650-443-2211		543 Dale Ave

The table is lexicographically sorted on the row keys

Cells

```
hbase(main):009:0> create 'guru99', {NAME=>'e', VERSIONS=>6}
0 row(s) in 1.3790 seconds

=> Hbase::Table - guru99
hbase(main):010:0> put 'guru99', 'r1', 'e:c1', 'value', 10
0 row(s) in 0.0880 seconds

hbase(main):011:0> put 'guru99', 'r1', 'e:c1', 'value', 12
0 row(s) in 0.0090 seconds

hbase(main):012:0> put 'guru99', 'r1', 'e:c1', 'value', 14
0 row(s) in 0.0110 seconds

hbase(main):013:0> delete 'guru99', 'r1', 'e:c1', 11
0 row(s) in 0.0270 seconds
```

Creating Table guru99 and placing values in guru99

```
hbase(main):010:0> get 'guru99','r1', 'c1'
COLUMN
c1:
1 row(s) in 0.0440 seconds

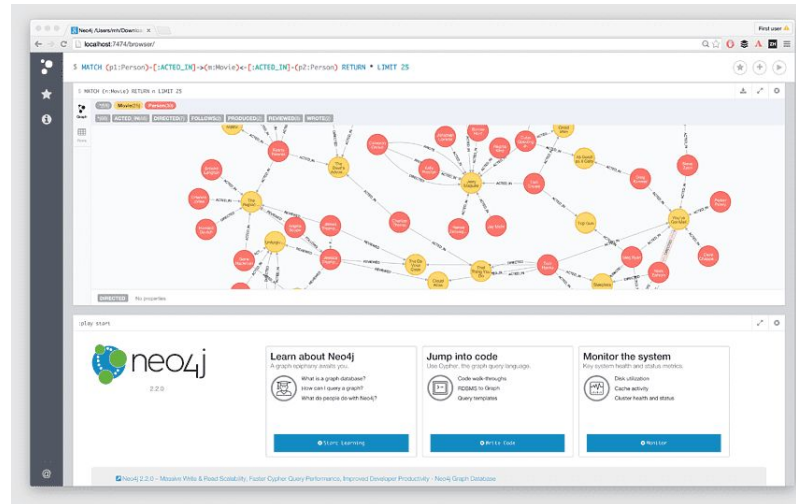
CELL
timestamp=30, value=value
```

getting records from 'guru99' using get

NoSQL Databases

Classification

- Graph databases :

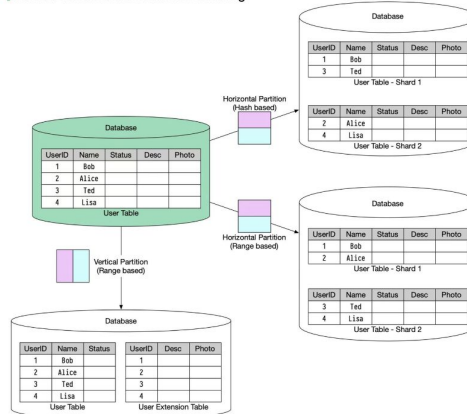


NoSQL Databases

Characteristics of databases

- **Data partitioning:**
 - Data divided into smaller, more manageable chunks (partitions/shards) based on a partitioning strategy.

Vertical & Horizontal Database Sharding



Key characteristics:

- Logical or physical separation of data based
- Each partition contains a subset of the complete dataset

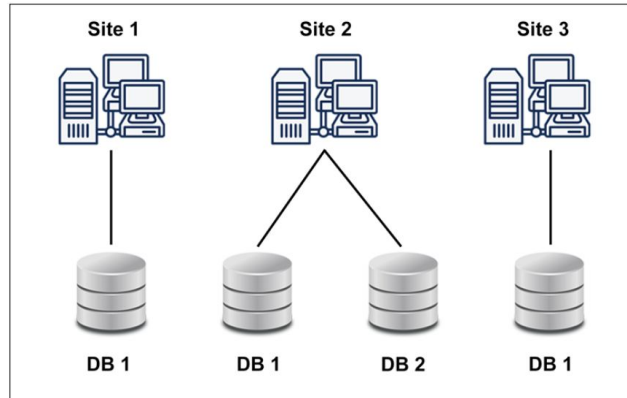
Purpose:

- Improved query performance
- More manageable data sizes
- Horizontal scalability

NoSQL Databases

Characteristics of databases

- **Data distribution**
 - Data is spread across multiple physical machines or nodes, typically in different locations.



Key characteristics:

- Geographic separation between nodes
- Replication of data across locations for availability/redundancy

Purpose:

- High availability
- Disaster recovery
- Geographic performance optimization



NoSQL Databases

Characteristics of databases

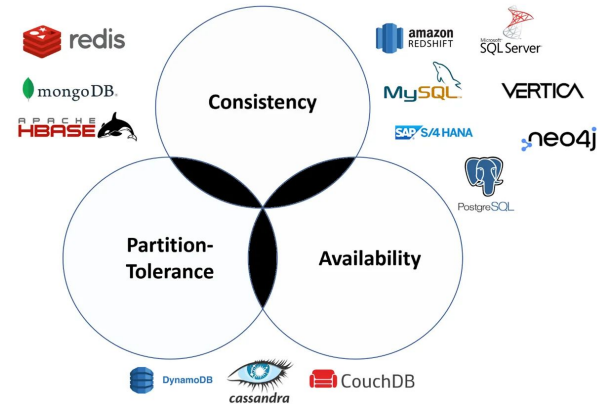
- **The CAP Theorem** => in any distributed system, it is not possible to satisfy these 3 properties at the same time :
 - **Consistency** : All servers see the same data at the same time (if you write something, everyone instantly sees it)
 - **Availability** : Every request gets a response, even if it's not the latest data (the system is always up and replies, even during problems)
 - **Partition tolerance** : The system still works even if parts of it can't communicate (like when a network cable is cut)
- **Rule** : We can only guarantee 2 out of the 3 at any moment, must sacrifice one if there's a failure (like a server crash or network problem)

NoSQL Databases

Characteristics of databases

- The CAP Theorem

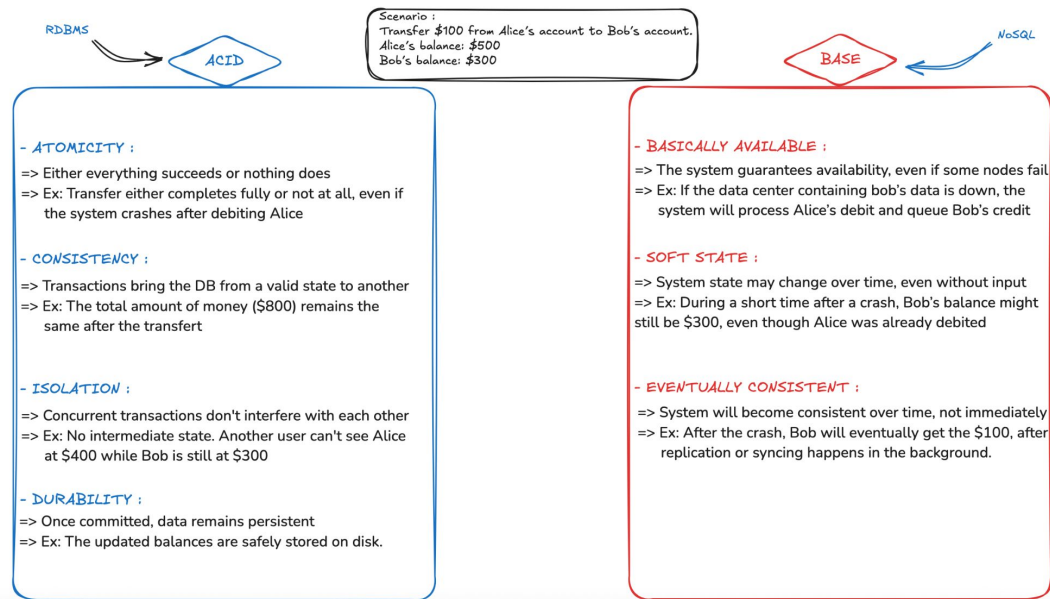
Choice	Example	Meaning
CP (Consistency + Partition)	HBASE, MongoDB	Always correct data, but might be unavailable during network issues
AP (Availability + Partition)	Amazon DynamoDB, Cassandra	Always responsive, but maybe outdated data sometimes
CA (Consistency + Availability)	Traditional databases (MySQL, PostgreSQL)	Needs perfect network – All clients can read and write on same data



NoSQL Databases

Characteristics of databases

- BASE Vs ACID





Part 3 :

Docker for Database
Environments



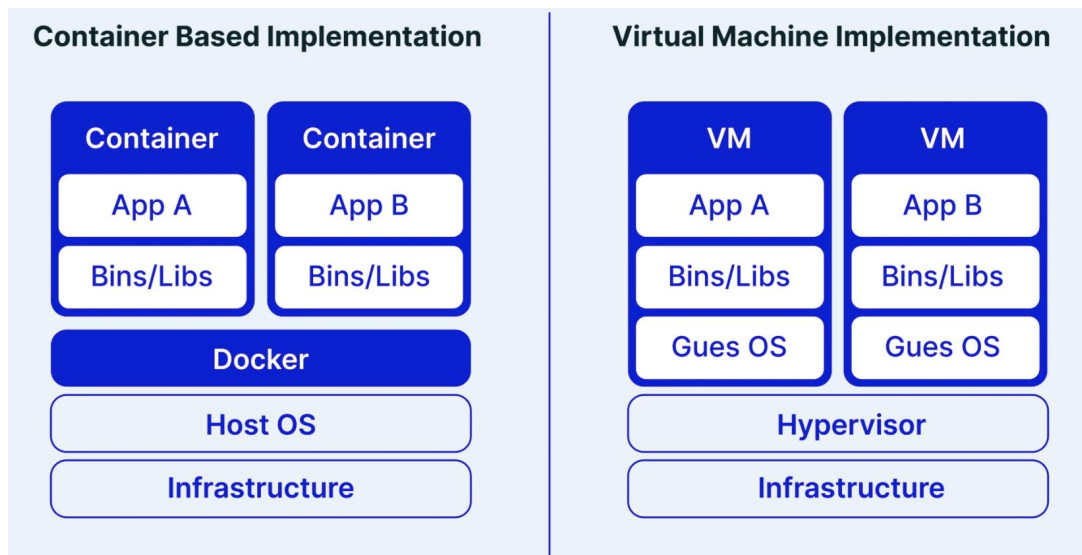
Docker for Database Environments

Introduction to containerization

- **Containerization** is a method of packaging and running applications along with their dependencies in a lightweight, portable, and isolated environment called a container
- Different from **virtualization** which is a process of creating an abstraction layer over hardware, allowing a single computer to be divided into multiple virtual computers.
- **Container** : standalone virtual machine, but without the overhead of a full Operating System. It includes:
 - The application (e.g., a database)
 - Its dependencies (libraries, binaries, config files)
 - Everything needed to run, regardless of the host system

Docker for Database Environments

Introduction to containerization





Docker for Database Environments

Containerization for databases

Why containerization in the context of databases ?

- Easy & Consistent Deployment, Portability
 - Containers work across different environments: Linux, Mac, Windows, cloud platforms.
 - The database will behave the same way on your machine, in staging, in production...
- Fast Testing and Development
 - Developers can test multiple versions of a database without complex installs.
 - Infrastructure as code: define your full DB setup in code and redeploy it anywhere.
 - You can isolate test environments easily.



Considerations for Production :

- Containers are **ephemeral**: by default, data is lost when the container is deleted.
- So for databases, it's critical to use volumes to persist data



Docker for Database Environments

Docker solution



What is Docker?

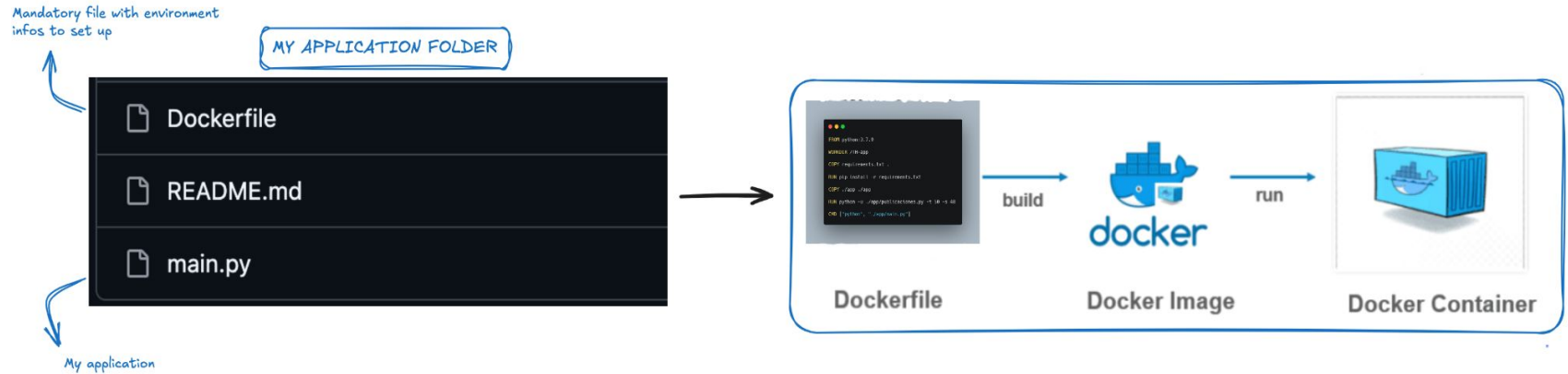
- An open-source platform that enables you to automate the deployment, scaling, and management of applications using lightweight & portable containers
- A Docker **container** bundles an application along with all its dependencies (libraries, configuration files, system tools) so it can run reliably across different computing environments

How Docker Works?

- A **Dockerfile** describes the environment and dependencies (possible to retrieve samples online)
- You build an image from that file
- You run that image as a container

Docker for Database Environments

Docker solution

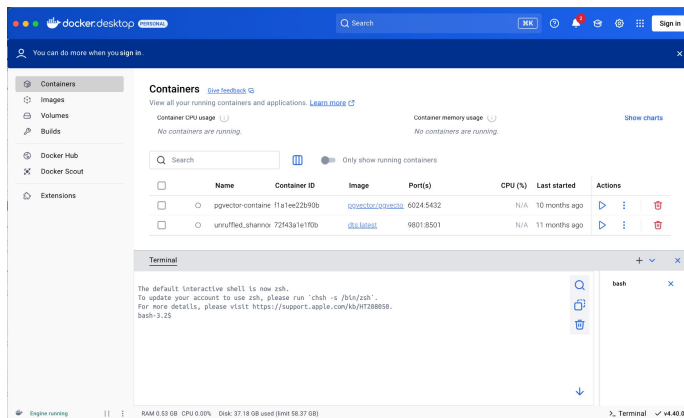


Docker for Database Environments

Installing docker

Desktop version :

- Windows : <https://docs.docker.com/desktop/setup/install/windows-install/>
- Mac OS : <https://docs.docker.com/desktop/setup/install/mac-install/>

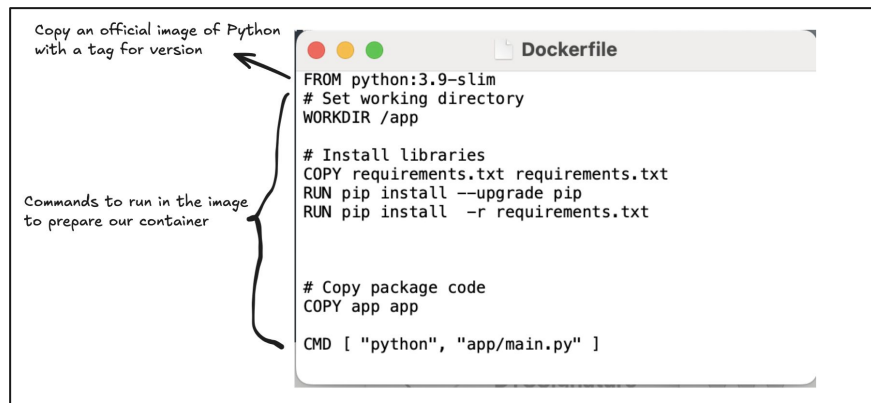


Docker for Database Environments

Building and running containers

Suppose we have a Python code (*main.py*) that we want to run inside a docker container.
We must complete **3 Steps** :

- 1- Create a **Dockerfile** in the same directory



Docker for Database Environments

Building and running containers

Suppose we have a Python code (*main.py*) that we want to run inside a docker container.
We must complete **2 Steps** :

- **2- Build** the Docker image

Open a terminal in the directory with your Dockerfile and run

```
bash
```

```
docker build -t my-python-app .
```

names the image

current directory as context

Docker for Database Environments

Building and running containers

Suppose we have a Python code (*main.py*) that we want to run inside a docker container.
We must complete **3 Steps** :

- **3- Run** the Docker image in a container

```
bash
docker run -it --name my-container my-python-app
```

gives a name to the container

name of the image to run

A diagram showing a terminal window with the command 'docker run -it --name my-container my-python-app'. A bracket under the text '--name my-container' is labeled 'gives a name to the container'. An arrow points from the text 'my-python-app' to the label 'name of the image to run'.

NB : the container will stop running after completing if the app is only about running a script



Docker for Database Environments

Building and running containers

Other docker commands

- **List containers:**
 - `docker ps`
- **Check logs:**
 - `docker logs my-container`
- **Stop a container:**
 - `docker stop my-container`
- **Remove a container:**
 - `docker rm my-container`
- **Copy a remote image from Docker hub :**
 - `docker pull `THE_OFFICIAL_IMAGE_NAME``



Docker for Database Environments

Building and running containers

Exercise

- **Basic docker app :**
 - Build & run your own docker image that executes a python script. This project can inspire you : https://github.com/ababacaryoro/nosql_course/tree/dev/Docker-init
- **Docker app with Jupyter notebook:**
 - Create a docker app running a Jupyter notebook application on **port 9988** of your computer
 - Mount with your app a local folder containing your notebooks when running your application



Docker for Database Environments

Docker Compose : multi-container applications

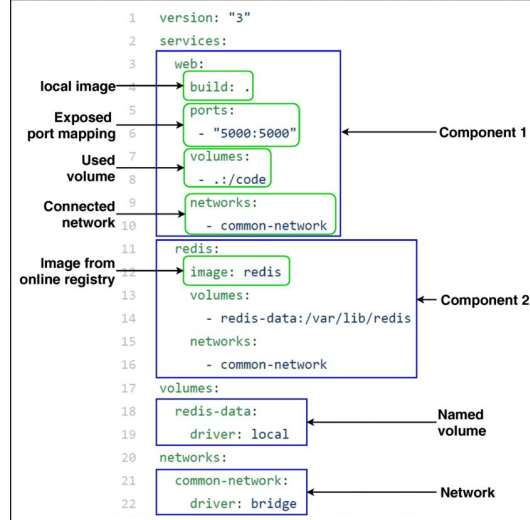
- When an application needs to run multiple services for different tasks => multiple docker applications
- Instead of running individual ***docker run*** commands for each service, Docker Compose lets you define them in a file called ***docker-compose.yml***
- Ex : a web app that needs a database, a cache, and maybe a background worker.
- Benefits :
 - Centralized configuration for all containers
 - Easy to bring up/down a multi-container setup
 - Networking between containers is automatic
 - Supports volume mounting, environment variables, port mappings

Docker for Database Environments

Docker Compose : multi-container applications

Steps to create a multi-container application

- 1- Create the *docker-compose.yml* file



Docker for Database Environments

Docker Compose : multi-container applications

Steps to create a multi-container application

- 2- Start the app
 - `docker-compose up` or `docker compose up` in MacOS
- Other commands :
 - Stop the App: `docker-compose down`
 - Build services from Dockerfiles : `docker-compose build`
 - View logs from services: `docker-compose logs`
 - List containers : `docker-compose ps`

Containers [Give feedback](#)

View all your running containers and applications. [Learn more](#)

Container CPU usage 0.01% / 1200% (12 CPUs available) Container memory usage 254.93MB / 7.48GB

Search

☐ Only show running containers

☐	Name	Container ID	Image	Port(s)
☐	pgvector-container	f1a1ee22890b	pgvector/pgvector:pg16	6024-5432
☒	multi-container	-	-	-
☐	db-1	5c66e6cb02c5	postgres:15	
☐	notebook-1	d717d4ae006a	jupyter/scipy-notebook	8888:8888



Docker for Database Environments

Docker Compose : multi-container applications

Exercise :

- Create a multi-container application running with a Jupyter notebook and a PostgreSQL app, with the following specifications :
 - Mount/create a volume for the folder of your notebooks with the Jupyter app
 - Create password & username with the PostgreSQL app (use a .env file)
 - Mount a persistent volume for the database
 - Run the container & open a new notebook with the Jupyter app
 - Using the notebook :
 - Connect to PostgreSQL and create a database “school”
 - Create 2 tables : students (columns name, birth date, class) and course (columns name, duration, level)
 - Add random data to theses tables
 - Check if was correctly added (load tables)



Part 4 :

Key-Value Stores with Redis



Key-Value Stores with Redis

Key-value store concepts

- A **Key-Value Store** is a simple type of NoSQL database where each piece of data is stored as a pair:
 - **Key** → Value
- **Structure :**
 - **Key:** Unique identifier (e.g., a string or number)
 - **Value:** Any kind of data (e.g., string, JSON, binary, image)
- **Example :**

```
"user:1001" → { "name": "Alice", "age": 29 }  
"session:xyz" → "abc123token"
```

Key-Value Stores with Redis

Key-value store concepts

Use cases





Key-Value Stores with Redis

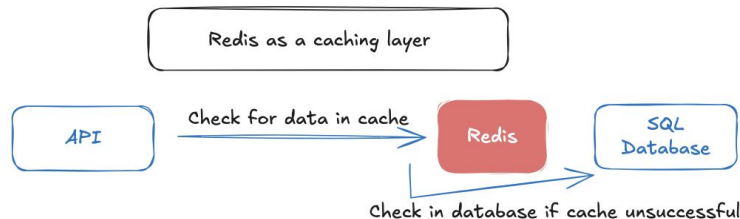
Key-value store concepts

- **Characteristics :**
 - **Very fast** due to $O(1)$ access
 - **Scalable** and simple to distribute across servers
 - **Easy to use** for developers
- **Limitations :**
 - No complex queries (e.g., JOINS)
 - Hard to model relationships
 - Not ideal for structured, relational data

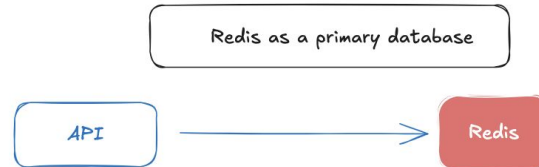
Key-Value Stores with Redis

Redis DB

- **Redis (Remote Dictionary Server)** : an open-source database management system designed to store, retrieve, and manipulate various key/value pair data
- Can be configured to store data on **disk**, however, it is primarily used for **in-memory** data management, making it much faster
- **Usage :**



Data is stored in memory => in case of crash, they will be lost in Redis but still available in the database

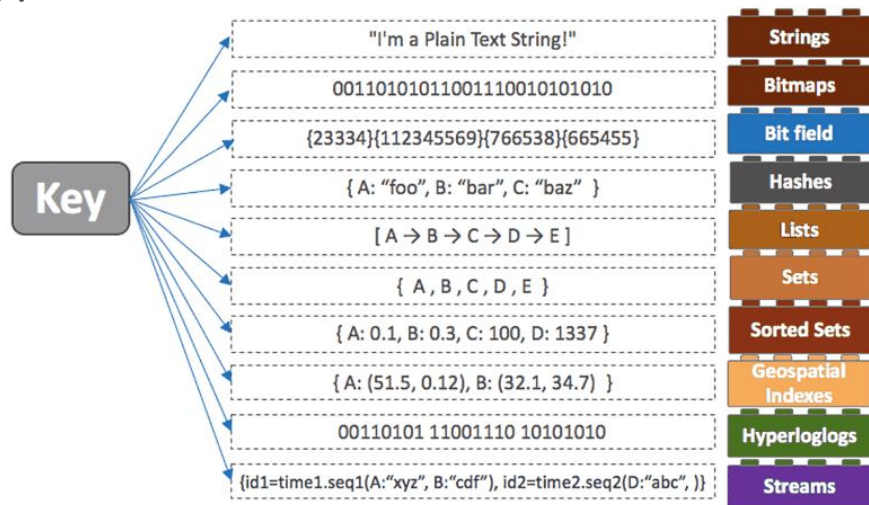


Data can be stored in disk => data is persisted and a system of replication is used to make data durable and available

Key-Value Stores with Redis

Redis DB

- Redis data types :





Key-Value Stores with Redis

Redis DB

- **Redis data organisation :**
 - No tables, collections or schema : just objects stores using keys/value system
- **Simulate common data organization :**
 - Use **data structures** and **keys naming conventions** to better manipulate data
- **Common Redis key naming conventions :**
 - Use namespacing with colon separators (:) -> 'user:123'; 'session:abc456'
 - Use lowercase letters and consistent casing : 'product:456' instead of 'Product:456'
 - Prefix by entity type or application : 'app1:user#1001'; 'app2:user#1001' (**colon** for namespaces, **sharp** for IDs)
 - Avoid spaces and special characters : Use underscores _ or colons ; instead.
- **TTL (Time To Live) :** defines how long a key should exist before it automatically expires and is deleted from the Redis database.
 - Can be defined when creating the (key, value)



Key-Value Stores with Redis

Hands-on Redis

- **Installing :**
 - Desktop client : <https://redis.io/open-source/>
 - Redis Cloud : <https://redis.io/cloud/>
 - Docker : https://redis.io/docs/latest/operate/oss_and_stack/install/install-stack/docker/
- **TODO :** install using docker compose
 - Install redis image => `image: redis:latest`
 - Add redis insight (GUI client) => `image: redis/redisinsight:latest`
 - Add Jupyter notebook



Key-Value Stores with Redis

Hands-on Redis

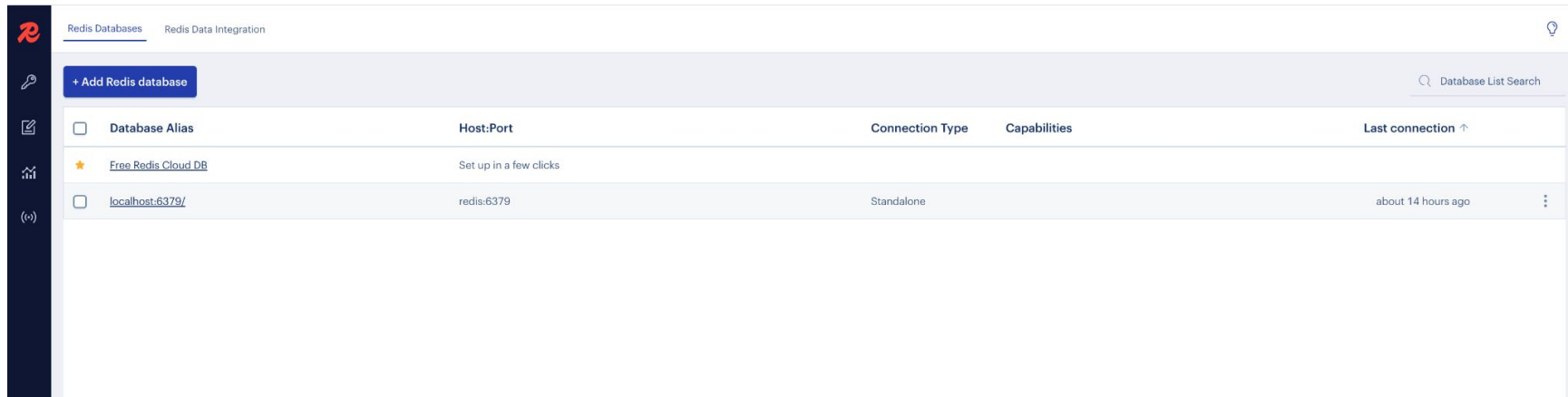
- **TODO** : install using docker compose

```
version: '3.9'
> Run All Services
services:
  > Run Service
  redis:
    image: redis:latest
    container_name: redis-server
    ports:
      - "6379:6379"
    volumes:
      - redis-data:/data
  > Run Service
  redisinsight:
    image: redis/redisinsight:latest
    container_name: redis-insight
    ports:
      - "5540:5540"
    depends_on:
      - redis
  > Run Service
  notebook:
    image: jupyter/scipy-notebook
    ports:
      - "9988:8888"
    volumes:
      - ./notebooks:/home/jovyan/work
    environment:
      - JUPYTER_ENABLE_LAB=yes
volumes:
  redis-data:
```

Key-Value Stores with Redis

Hands-on Redis

- Redis insight GUI client



The screenshot displays the Redis Insight web interface. On the left is a dark sidebar with navigation icons. The main content area has a header with 'Redis Databases' and 'Redis Data Integration' tabs. Below the header is a '+ Add Redis database' button and a 'Database List Search' input field. A table lists the configured Redis databases with columns for 'Database Alias', 'Host:Port', 'Connection Type', 'Capabilities', and 'Last connection'. Two databases are listed: 'Free Redis Cloud DB' and 'localhost:6379/'.

Database Alias	Host:Port	Connection Type	Capabilities	Last connection ↑
★ Free Redis Cloud DB	Set up in a few clicks			
☐ localhost:6379/	redis:6379	Standalone		about 14 hours ago

Key-Value Stores with Redis

Hands-on Redis

- Redis insight GUI client

Tab to visualize data

Tab for queries

Query	Timestamp	Duration	Actions
HSET person:1 name Alice	02-02-17 4 May 2025	19.537 msec	Clear Results
HGET user:112	02-01-41 4 May 2025	3.826 msec	Clear Results
GET user:112	02-01-25 4 May 2025	4.236 msec	Clear Results
HSET user:112 name Alice age 3	02-01-01 4 May 2025	1.963 msec	Clear Results

Key-Value Stores with Redis

Hands-on Redis

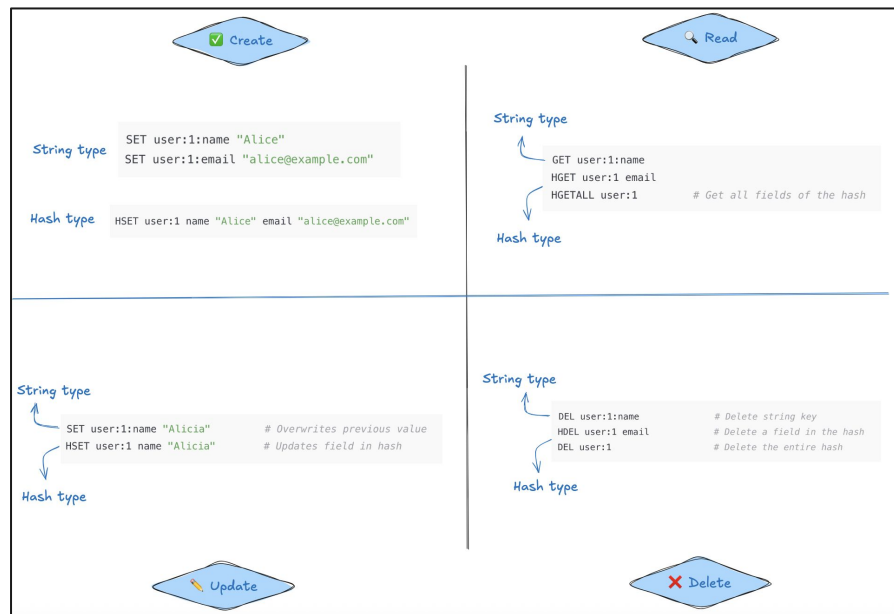
- **Basic commands :** Each data structure in Redis has its own unique set of operations that can be performed on it :

Category	Command	Description	Example
🔑 Keys	SET key value	Store a value	SET user:1 "Alice"
	GET key	Retrieve a value	GET user:1
	DEL key	Delete a key	DEL user:1
	EXISTS key	Check if key exists	EXISTS user:1
	EXPIRE key seconds	Set TTL	EXPIRE user:1 60
	TTL key	Get remaining TTL	TTL user:1
📄 Strings	APPEND key value	Add to string	APPEND user:1 " Smith"
🔢 Incr	INCR key / DECR key	Increment/Decrement	INCR counter
🗑 Hashes	HSET key field value	Set hash field	HSET user:1 name Alice
	HGET key field	Get hash field	HGET user:1 name
📋 Lists	LPU SH key value	Add to list start	LPU SH todos "buy milk"
	LRANGE key 0 -1	Get all list items	LRANGE todos 0 -1
🔢 Sets	SADD key value	Add to set	SADD tags redis
	SMEMBERS key	Get set members	SMEMBERS tags

Key-Value Stores with Redis

Hands-on Redis

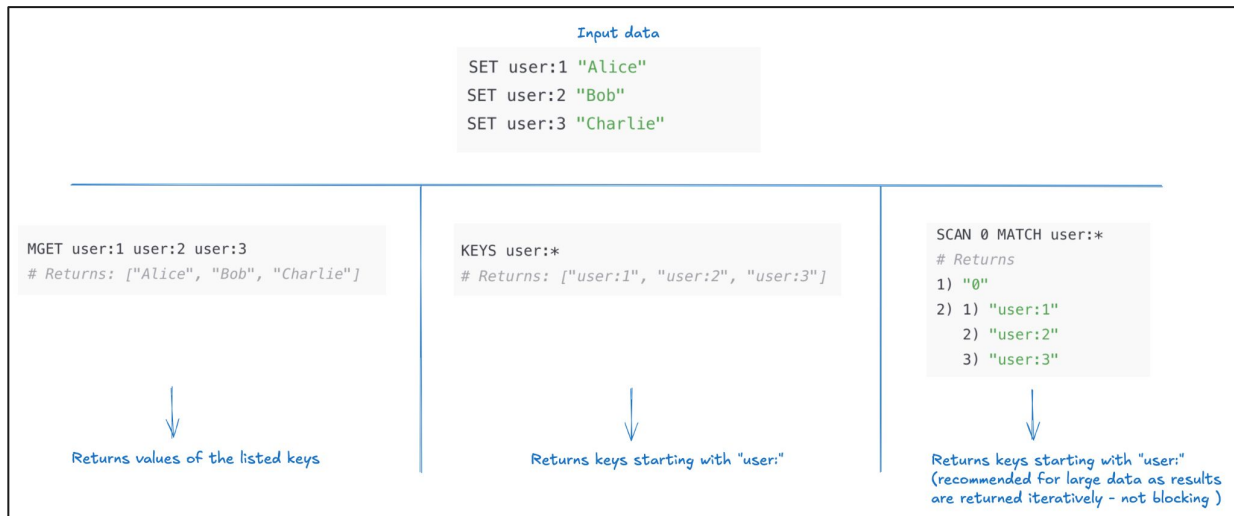
- CRUD (Create, Read, Update, Delete):



Key-Value Stores with Redis

Hands-on Redis

- Retrieve multiple keys :



Key-Value Stores with Redis

Hands-on Redis

- Basic commands : using Python client



```
!pip install redis
import redis
r = redis.Redis(host='redis', port=6379, db=0)
```

Create

```
# Set a key
r.set("user:1", "Alice")

# Set multiple keys
r.mset({"user:2": "Bob", "user:3": "Charlie"})
```

Read

```
# Get a value
r.get("user:1") # b'Alice'

# Get multiple values
r.mget(["user:1", "user:2"]) # [b'Alice', b'Bob']

# List all matching keys
r.keys("user:*") # [b'user:1', b'user:2', b'user:3']
```

Update

```
# Overwrite value
r.set("user:1", "Alice Smith")

# Increment value (e.g., score)
r.set("score", 100)
r.incr("score") # 101
r.incrby("score", 5) # 106
```

Delete

```
# Delete a single key
r.delete("user:1")

# Delete multiple keys
r.delete("user:2", "user:3")
```



Key-Value Stores with Redis

Hands-on Redis

- **Application :**
 - Instructions : https://github.com/ababacaryoro/nosql_course/blob/dev/Redis/redis_practical_exercise.md
 - Actions to do :
 - Load data into Redis using Docker
 - Interact with Redis using Python
 - Write queries to analyze data